

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)No Output
- B)A, followed by propagation of the same exception
- C)A, and execution ends normally
- D)B

Q2. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")
```

```
print(fun())
```

- A)A
- C
- D
- 20
- B)A
- B
- 10
- D
- C)A
- B
- D
- 10

D)A

B

C

D

10

Q3. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

```
print(fun(3))
```

- A)3
- B)6
- C)5
- D)None

Q4. What happens here?

```
def fun(a):
    return a
```

```
print(fun())
```

- A)0
- B)TypeError
- C)NameError
- D)None

Q5. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

- A)Error



B)10

Hello

C)x

display

D)Only Hello

Q6. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)9

B)24

C)0

D)10

Q7. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)50

B)NameError

C)UnboundLocalError

D)10

Q8. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)ZeroDivisionError is not an exception

B)ZeroDivisionError occurs before Exception

C)except blocks execute randomly

D)Exception is a superclass of ZeroDivisionError

Q9. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A C

B)A

C)B C

D)B

Q10. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)B C

B)B

C)A C

D)A

Q11. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)10

B)SyntaxError

C)20

D)NameError

Q12. What happens when this code runs?



```
def first():
    second()

def second():
    third()

def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

A)NameError
B)Handled
C)RuntimeError cannot propagate through functions
D)TypeError

Q13. What is the output?

```
def outer():
    x = 10

    def inner():
        return x + 5

    return inner
```

```
f = outer()
print(f())
```

A)10
B)NameError
C)None
D)15

Q14. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

A)Nothing
B)ValueError
C)e
D)Wrong Value

Q15. What is the output?

```
def fun(x):
```

```
x += 10
return x
```

```
x = 5
y = fun(x)
```

```
print(x, y)
```

A)5 5
B)15 5
C)15 15
D)5 15

Q16. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
b = fun()
```

```
print(a)
print(b)
```

A)[10]
[10]
B)[10, 10]
[10]
C)[10]
[10, 10]
D>Error

Q17. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

A)There is no difference
B)First imports only sqrt
C)First provides the module namespace; second imports sqrt directly
D)Both import only sqrt

Q18. What will normally happen?

```
import math
import math
```

A)ImportError



B)The module is normally loaded and executed twice
C)SyntaxError
D)The module is normally loaded once and reused

Q19. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)NameError
B)36
C)6
D)6.0

Q20. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

A)30
B)10
C)20
D)-10

Q21. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)TypeError
B)30
C)20
D)SyntaxError

Q22. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

A)1

2

3

4

B)1

2

1

3

C)0

1

0

2

D)1

1

1

1

Q23. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

A)A
B)Nothing
C)A B
D)B

Q24. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

A)25
B)TypeError
C)15
D)SyntaxError

Q25. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```



- A)10
- B)15
- C)Error
- D)5

Q26. What will be the output?

```
def fun(x):
    try:
        if x < 0:
            raise ValueError("negative")
        return x
    except ValueError:
        return 100
    finally:
        print("cleanup")
```

print(fun(-5))

- A)cleanup
- 5
- B)100
- cleanup
- C)negative
- cleanup
- D)cleanup
- 100

Q27. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)Module loaded
- 10
- B)ImportError
- C)Module loaded
- Module loaded
- 10

- D)10
- 10

Q28. What will be printed?

```
def fun(a, b):
    return a + b
```

print(fun(10, b=20))

- A)TypeError
- B)30
- C)SyntaxError
- D)20

Q29. if test.py has
print("A")

```
if __name__ == "__main__":
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A
- B)A followed by B
- C)B
- D)Nothing

Q30. What is the output?

```
def fun(x):
    return x * 2
```

```
a = fun
print(a(5))
```

- A>Error
- B)10
- C)5
- D)func(5)

Q31. What will be the output?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        global x
        x = 30
```

```
    inner()
    print(x)
```



```
outer()
print(x)
```

- A)20
- 30
- B)20
- 10
- C)30
- 30
- D)30
- 20

Q32. What is the output?

```
def fun():
    try:
        return 10
    finally:
        return 20
```

```
print(fun())
```

- A)RuntimeError
- B)None
- C)20
- D)10

Q33. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)
```

```
print(result)
```

- A)[2, 4]
- B)[4, 8]
- C)[2, 4, 6, 8, 10]
- D)[1, 3, 5]

Q34. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)Nothing
- B)B
- C)A
- D)A and B

Q35. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

- A)Finally followed by 10
- B)10 only
- C)Finally only
- D)10 followed by Finally

Q36. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)Python automatically renames the local file
- B)random cannot be imported
- C)The local file can shadow the intended standard-library module
- D)Python will always reject it

Q37. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A)NameError
- B)None
- C)20
- D)10

Q38. What will happen?



```
def fun(n):
    if n == 0:
        return 1
    return n * fun(n)
```

print(fun(3))

A) RecursionError

B) 6

C) TypeError

D) 3

Q39. What will be printed?

```
try:
```

```
    print(10 / 0)
```

```
except ZeroDivisionError:
```

```
    print("A")
```

```
except Exception:
```

```
    print("B")
```

A) A

B) A B

C) Nothing

D) B

Q40. What will be printed?

```
class InvalidAgeError(Exception):
    pass
```

```
def check_age(age):
```

```
    if age < 18:
```

```
        raise InvalidAgeError("Age is
too low")
```

```
    return "Allowed"
```

```
try:
```

```
    print(check_age(15))
```

```
except InvalidAgeError as e:
```

```
    print(type(e).__name__)
```

```
    print(e)
```

A) Nothing, because custom exceptions cannot be caught

B) InvalidAgeError

15

C) Exception

Age is too low

D) InvalidAgeError

Age is too low

Q41. What does the following function return?

```
def fun():
```

```
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A) Error

B) Hello None

C) Hello

D) None

Q42. Which object keeps track of modules that have already been imported during the current Python process?

A) sys.imports

B) sys.path

C) __module__

D) sys.modules

Q43. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A) 20

B) NameError

C) 30

D) 10

Q44. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A) sqrt(25)

B) 25

C) 5

D) 5.0



Q45. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

- A)NameError
- B)UnboundLocalError
- C)10
- D)20

Q46. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It identifies code intended to run when the file is executed directly
- B)It prevents all imports
- C)It creates the main function
- D)It prevents syntax errors

Q47. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

- A)10
- B)UnboundLocalError
- C)5
- D)15

Q48. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

- A)SyntaxError
- B)NameError
- C)It executes correctly

D)TypeError

Q49. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

- A)SyntaxError
- B)A
- C)A B
- D)B

Q50. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

- A)TypeError
- B)MyError
- C)Exception
- D)Something went wrong

